

get_log_probs — Step-by-Step Explanation

JR

February 22, 2026

Contents

1	Overview	2
2	Inputs and Output	2
3	The Core Alignment Idea	2
4	Diagram 1 — Sequence Alignment and Slicing	2
5	Diagram 2 — The gather Operation	3
6	Step-by-Step Walkthrough	3
6.1	Step 1 — Convert logits to log probabilities	3
6.2	Step 2 — Drop the last position	4
6.3	Step 3 — Get the actual next tokens	4
6.4	Step 4 — Add a trailing dimension for gather	4
6.5	Step 5 — Select the log prob for the correct token	4
6.6	Step 6 — Remove the trailing size-1 dimension	4
7	Concrete Example	4
8	Why Log Probabilities?	5

1 Overview

`get_log_probs` is a utility function used during transformer training. Given the model's raw output (logits) and the actual token sequence, it returns the **log probability the model assigned to each correct next token**.

```
1 def get_log_probs(  
2     logits: Float[Tensor, "batch posn d_vocab"],  
3     tokens: Int[Tensor, "batch posn"]  
4 ) -> Float[Tensor, "batch posn-1"]:  
5     log_probs = logits.log_softmax(dim=-1)  
6     log_probs_for_tokens = (  
7         log_probs[:, :-1]  
8         .gather(dim=-1, index=tokens[:, 1:].unsqueeze(-1))  
9         .squeeze(-1)  
10    )  
11    return log_probs_for_tokens
```

2 Inputs and Output

Argument	Shape	Meaning
logits	[batch, posn, d_{vocab}]	Raw unnormalized scores for every vocab token at every position
tokens	[batch, posn]	Actual token IDs in the sequence
return	[batch, posn - 1]	Log prob of the correct next token at each position

3 The Core Alignment Idea

A transformer at position i predicts the token at position $i + 1$. This means the model's output at position i must be evaluated against the *next* token in the sequence, not the current one.

Consider the sequence ["The", "cat", "sat", "."] with token IDs [1, 2, 3, 4]:

Position	0	1	2	3
Token	"The"	"cat"	"sat"	"."
Model predicts →	"cat"	"sat"	"."	(no target)

The last position has no ground-truth next token to score against, so the output has shape `posn - 1` rather than `posn`.

4 Diagram 1 — Sequence Alignment and Slicing

The two slicing operations (`log_probs[:, :-1]` and `tokens[:, 1:]`) create a matched pair of tensors: every prediction position lines up with its ground-truth target. Red cells are discarded; blue cells are prediction blocks; green cells are target tokens.

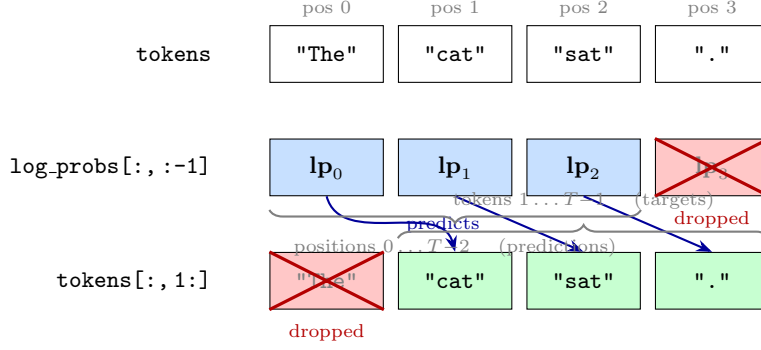


Figure 1: Slicing alignment. Each blue prediction block lp_i is evaluated against the green target token at position $i+1$. The red cells (last prediction, first token) are discarded because they have no counterpart.

5 Diagram 2 — The gather Operation

After slicing, log_probs[:, :-1] is a matrix of shape $[\text{posn}-1, d_{\text{vocab}}]$. **gather** selects **one cell per row** — the log probability of the actual next token — and assembles them into the output vector.

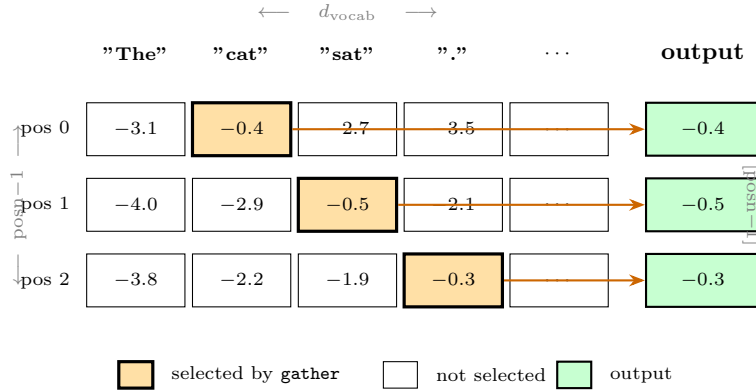


Figure 2: The **gather** operation. Each row of log_probs[:, :-1] holds log probabilities over all d_{vocab} tokens. **gather** picks the **highlighted** (orange) cell — the log prob of the true next token — and writes it to the output vector. The final **squeeze(-1)** collapses the trailing size-1 dimension, yielding shape $[\text{batch}, \text{posn}-1]$.

6 Step-by-Step Walkthrough

6.1 Step 1 — Convert logits to log probabilities

```
1 log_probs = logits.log_softmax(dim=-1)
2 # shape: [batch, posn, d_vocab] (unchanged)
```

log_softmax is applied along $\text{dim}=-1$ (the vocab dimension). For each position p in batch item b , it computes:

$$\text{log_probs}[b, p, v] = \log \frac{e^{\text{logits}[b, p, v]}}{\sum_{v'} e^{\text{logits}[b, p, v']}} = \log P(\text{token}_v \mid \text{context up to } p)$$

Using log_softmax instead of $\text{softmax} + \text{log}$ is numerically stable because it avoids computing very small probabilities before taking the log.

6.2 Step 2 — Drop the last position

```
1 log_probs[:, :-1]
2 # shape: [batch, posn-1, d_vocab]
```

The model at the last position has no next token to be evaluated against, so we discard it. Only positions $0, 1, \dots, \text{posn} - 2$ remain.

6.3 Step 3 — Get the actual next tokens

```
1 tokens[:, 1:]
2 # shape: [batch, posn-1]
```

We drop the first token and keep the rest. These are the ground-truth *targets*: the token at index 1 is what position 0 should have predicted, the token at index 2 is what position 1 should have predicted, and so on.

The alignment is now:

$$\underbrace{\text{log_probs[:, :-1]}}_{\text{predictions from positions } 0 \dots \text{posn}-2} \longleftrightarrow \underbrace{\text{tokens[:, 1 :]} }_{\text{targets at positions } 1 \dots \text{posn}-1}$$

6.4 Step 4 — Add a trailing dimension for gather

```
1 tokens[:, 1:].unsqueeze(-1)
2 # shape: [batch, posn-1, 1]
```

`torch.gather` requires the *index* tensor to have the same number of dimensions as the *source* tensor. The source `log_probs[:, :-1]` has 3 dimensions $[\text{batch}, \text{posn} - 1, d_{\text{vocab}}]$, so we add a trailing dimension of size 1 to the index tensor.

6.5 Step 5 — Select the log prob for the correct token

```
1 log_probs[:, :-1].gather(dim=-1, index=tokens[:, 1:].unsqueeze(-1))
2 # shape: [batch, posn-1, 1]
```

`gather(dim=-1, index=idx)` picks, for each (b, p) slot, exactly one value along the vocab dimension:

$$\text{output}[b, p, 0] = \text{log_probs}[b, p, \underbrace{\text{tokens}[b, p + 1]}_{\text{true next token ID}}] = \log P(\text{true next token} \mid \text{context})$$

6.6 Step 6 — Remove the trailing size-1 dimension

```
1 .squeeze(-1)
2 # shape: [batch, posn-1]
```

`squeeze(-1)` removes the trailing dimension of size 1 introduced by `unsqueeze` and `gather`, giving the clean output shape.

7 Concrete Example

Let $\text{batch} = 1$, $\text{sequence} = [\text{"The"}, \text{"cat"}, \text{"sat"}]$, token IDs = $[1, 2, 3]$.

Tensor	What it contains
<code>log_probs</code>	shape $[1, 3, d_{\text{vocab}}]$; log probs at all 3 positions
<code>log_probs[:, :-1]</code>	shape $[1, 2, d_{\text{vocab}}]$; positions 0 and 1 only
<code>tokens[:, 1:]</code>	$[[2, 3]]$; “cat” and “sat” are the targets
<code>gather output</code>	picks <code>log_probs[0,0,2]</code> and <code>log_probs[0,1,3]</code>
<i>result</i>	$[\log P(\text{"cat"} \mid \text{"The"}), \log P(\text{"sat"} \mid \text{"The cat"})]$

8 Why Log Probabilities?

- **Numerical stability.** Raw probabilities of long sequences become vanishingly small; working in log space avoids underflow.
- **Summability.** The log probability of an entire sequence factorises as a sum:

$$\log P(x_1, x_2, \dots, x_T) = \sum_{t=1}^T \log P(x_t \mid x_{<t})$$

- **Direct connection to cross-entropy loss.** Averaging the negated log probs over the sequence gives the standard language-modelling cross-entropy loss:

$$\mathcal{L} = -\frac{1}{T-1} \sum_{t=0}^{T-2} \log P(x_{t+1} \mid x_{\leq t})$$